

Knowledge Graph Extraction Pipeline

A Detailed Report on the Historical NLP Research System

Executive Summary

This pipeline transforms historical PDF books into an interactive knowledge graph to investigate whether salt routes in the Western Himalayas formed the structural backbone of multi-commodity trade networks and frontier governance in Colonial North India. It reads books automatically, identifies the people, places, goods, and institutions mentioned in them, discovers how those entities relate to one another, and produces a visual, explorable map of those connections. Each book processed adds to a single growing graph that accumulates knowledge across the entire research corpus.

1 Background and Research Context

The research focuses on Himalayan frontier regions — Ladakh, Tibet, Rupshu, Changthang, Kashmir, Humla, Dolpo, Zaskar, and neighboring areas — during the colonial period. The key subjects of interest include:

- **Commodities:** salt, pashm (cashmere wool), barley, tea, butter, grain, livestock, musk, rice
- **Actors:** pastoral communities, merchants, colonial officials, frontier states, monasteries, local institutions
- **Dynamics:** trade routes, barter networks, taxation, licensing, sovereignty, monopolies, political control

The primary sources are historical texts — gazetteers, travel accounts, administrative records, reports, and historical monographs — available as PDF files. These are rich in information but are entirely unstructured: their knowledge lives in narrative prose, not in tables or databases. Extracting structured, comparable data from them by hand would take years. This pipeline automates that extraction.

2 The Big Picture: What the Pipeline Does

At the highest level, the pipeline does three things:

1. **Reads** a PDF book and converts it into a form a computer can analyze.
2. **Finds** named entities (people, places, goods, groups) and understands how they appear near each other in the text.
3. **Understands** the relationships between those entities — who trades what with whom, who controls whom, who taxes whom — and stores those relationships as a structured graph.

The result is a **knowledge graph**: a web of nodes (entities) connected by labeled edges (relationships). This graph can be explored interactively in a browser, searched, filtered, and used to answer research questions that would be extremely difficult to investigate by reading alone.

The entire process is managed by a single orchestrator program (`pipeline.py`) that runs all the stages in the correct order automatically, meaning a researcher only needs to point the system at a book and it handles the rest.

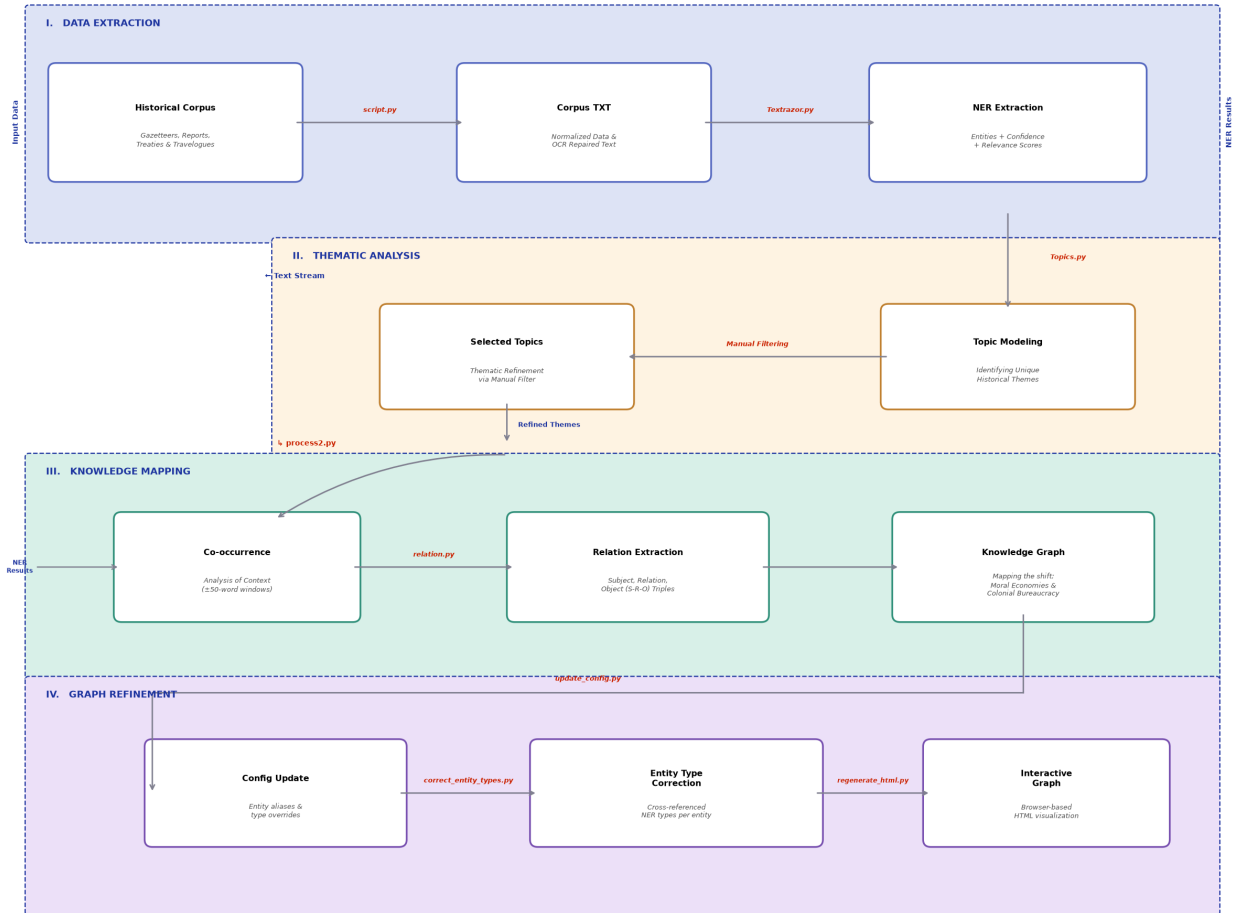


Fig 2. Archival-to-Network Methodological Pipeline for Analysing Himalayan Salt Trade.

Figure 1: Archival-to-Network Methodological Pipeline Architecture Diagram

3 The Pipeline Stages — A Walkthrough

3.1 Stage 1: Reading the Book — PDF to Plain Text

What happens: The first step is simply getting the text out of the PDF. A PDF is not plain text; it is a formatted document with fonts, layouts, and image layers. The pipeline uses a library to open the PDF, go through every single page, and extract the raw written words, saving them into a plain text file.

Why it matters: Every subsequent stage depends on being able to read and analyze the text. This stage is the foundation. The text is saved to a shared "corpus" folder so it can be reused without reprocessing the PDF.

Output: A `.txt` file containing the full text of the book, one paragraph per section, stored in

the `corpus/` directory.

3.2 Stage 2: Finding What Matters — Named Entity Recognition and Topic Discovery

What happens: This is the most powerful early step. The entire text is sent to a cloud-based AI service called **TextRazor**, which specializes in understanding language and identifying meaningful things within it.

TextRazor does two things simultaneously:

- **Named Entity Recognition (NER):** It reads through the text and highlights every entity — every person’s name, place name, organization, commodity, and concept — and classifies each one. It also assigns a relevance score and a confidence score to each entity, indicating how certain it is and how central to the text it seems to be.
- **Topic Classification:** It identifies the broad thematic topics present in the text, using a categorization system called Freebase (a large knowledge taxonomy). So it might determine that the text contains content about `/food/food`, `/government/governmental_jurisdiction`, `/people/ethnicity`, and so on.

Because historical books can be very long, the text is divided into chunks before being sent to TextRazor, to stay within the service’s data limits. Each chunk is processed separately, and the results are combined.

Output: A `ner_results.txt` file that lists every entity found in the book, grouped by category (e.g., all entities that fall under "Person", all entities under "Location"), along with their relevance and confidence scores. It also contains the thematic topics detected in the text.

3.3 Stage 3: What the Book is Really About — Topic Aggregation

What happens: The NER stage produces a very long, detailed list of topics — potentially hundreds of them, many irrelevant to the specific research. The pipeline runs a second step that reads through the `ner_results.txt` file, pulls out every unique topic label, removes duplicates, and sorts them alphabetically.

Output: A `unique_topics.txt` file listing every distinct thematic topic detected across the book.

The Human Step — Topic Curation: This is where a researcher steps in. The `unique_topics.txt` file will contain relevant topics (like `/food/food` or `/location/region`) but also many irrelevant ones (like `/astronomy/galaxy` or `/amusement_parks/ride_theme`). The researcher reviews the list and manually creates a filtered file called `Selected_Topics.txt`, keeping only the topics relevant to the research.

This human curation step is critical — it is what focuses all subsequent analysis on historically meaningful entities and prevents the graph from being polluted with noise. The `Selected_Topics.txt` file acts as a **research lens** through which all further processing is done.

3.4 Stage 4: Who Appears Near Whom — Co-occurrence Analysis

What happens: Now the pipeline knows which topics are relevant (from `Selected_Topics.txt`) and which entities belong to those topics (from `ner_results.txt`). In this stage, it goes back to the raw text of the book and performs a sweep to find **co-occurrences**: instances where two relevant entities appear close to each other.

The logic works like this:

```

/food/food
Apricot (Relevance 0.061 | Confidence 2.727)
Barley (Relevance 0.232 | Confidence 11.420)
Butter tea (Relevance 0.598 | Confidence 2.376)
Daikon (Relevance 0.107 | Confidence 2.237)
Grain (Relevance 0.045 | Confidence 4.496)
Milk (Relevance 0.195 | Confidence 5.863)
Radish (Relevance 0.041 | Confidence 3.142)
Rice (Relevance 0.172 | Confidence 11.090)
Salt (Relevance 0.267 | Confidence 7.832)
Sheep (Relevance 0.320 | Confidence 11.590)
Spice (Relevance 0.216 | Confidence 2.853)
Sugar (Relevance 0.097 | Confidence 8.269)
Tea (Relevance 0.247 | Confidence 5.364)
Walnut (Relevance 0.022 | Confidence 1.993)

/food/ingredient
Apricot (Relevance 0.061 | Confidence 2.727)
Barley (Relevance 0.232 | Confidence 11.420)
Blood (Relevance 0.171 | Confidence 5.064)
Daikon (Relevance 0.107 | Confidence 2.237)
Milk (Relevance 0.195 | Confidence 5.863)
Radish (Relevance 0.041 | Confidence 3.142)
Rice (Relevance 0.172 | Confidence 11.090)
Salt (Relevance 0.267 | Confidence 7.832)
Sea salt (Relevance 0.080 | Confidence 1.689)
Sheep (Relevance 0.320 | Confidence 11.590)
Spice (Relevance 0.216 | Confidence 2.853)
Sugar (Relevance 0.097 | Confidence 8.269)
Tea (Relevance 0.247 | Confidence 5.364)
Walnut (Relevance 0.022 | Confidence 1.993)

/food/nutrient
Sugar (Relevance 0.097 | Confidence 8.269)

/geography/body_of_water
Indus River (Relevance 0.237 | Confidence 7.177)
Tso Kar (Relevance 1.000 | Confidence 1.222)
Tso Moriri (Relevance 0.475 | Confidence 1.584)

```

Figure 2: Automated Entity Extracted Categorization and Scores before Human Curation

- The pipeline selects a set of **”anchor” entities** — entities that belong to the researcher’s chosen topics. These are the central subjects of interest (e.g., ”salt”, ”Ladakh”, ”Tibet”, ”Changpa”).
- It then scans the full text word by word. Every time it spots one of these anchor entities, it looks at a window of roughly 50 words on either side — roughly a paragraph of context — and records every other recognized entity that appears in that window.
- This is repeated across the entire text, and a **co-occurrence count** is built up: how many times did Entity A and Entity B appear near each other in the text?

Co-occurrence is not the same as a relationship. It simply means the two entities are mentioned in similar contexts. But it is a powerful signal: if ”salt” and ”Ladakh” and ”taxation” appear near each other repeatedly, there is likely a meaningful historical relationship worth investigating.

Output: An `entity_cooccurrences.txt` file that lists each anchor entity, the other entities that appear near it, how many times they co-occurred, and an example snippet of the actual text where the co-occurrence was found.

```
Anchor Entity: salt [/food/ingredient]
Example Context:
...The Salt Trade: Rupshu's Annual Trek to Tso Kar Monisha Ahmed It was growing dark in Tharchen's stone-walled store-house at Thugje, but preparations
Co-occurring Entities (sorted by frequency):
↳ tso [/location/location] - 58 times
↳ kar [/location/location] - 43 times
↳ rupshupa [/organization/organization] - 40 times
↳ lake [/geography/lake_type] - 37 times
↳ trade [/organization/organization_sector] - 36 times
↳ tibet [/travel/travel_destination] - 28 times
↳ rupshu [/location/location] - 27 times
↳ ladakh [/travel/travel_destination] - 19 times
↳ sheep [/visual art/art_subject] - 15 times
↳ tent [/business/industry] - 12 times
↳ beall [/people/person] - 11 times
↳ paper [/visual art/visual art support] - 9 times
↳ baba [/location/location] - 9 times
↳ goldstein [/people/person] - 9 times
↳ livestock [/business/industry] - 8 times
↳ angchuk [/people/person] - 7 times
↳ kharnak [/location/statistical region] - 7 times
↳ ocean [/symbols/namesake] - 7 times
↳ kashmir [/location/statistical region] - 6 times
```

Figure 3: Co-occurrence Entities and Frequencies for Anchor Entity 'Salt'

3.5 Stage 5: What Is Actually Happening — Relation Extraction with a Local AI

What happens: This is the most intellectually ambitious stage. The pipeline takes every co-occurring entity pair from the previous step and tries to determine the **precise relationship** between them, expressed as a structured triple: **Subject** → **Relation** → **Object**.

For example, instead of just knowing that ”Ladakhi merchants” and ”salt” appear near each other, the system tries to determine the exact nature of that connection — perhaps **Ladakhi merchants** → **trades_with** → **salt** or **salt** → **extracted_from** → **Tso Kar lake**.

To do this, the pipeline uses a **local Large Language Model (LLM)** running on the researcher’s own machine — specifically, **Llama 3** running through a tool called **Ollama**. This means the analysis is private and does not send sensitive research data to any external cloud service.

For each entity pair, the pipeline:

1. Finds the actual sentences in the book where both entities appear together.
2. Sends those sentences to Llama 3 with strict instructions: ”Read this text carefully. What is the relationship between these two entities? Choose from this specific list of allowed relationships only. Provide your answer as structured data and include the exact sentence from the text that justifies your answer.”

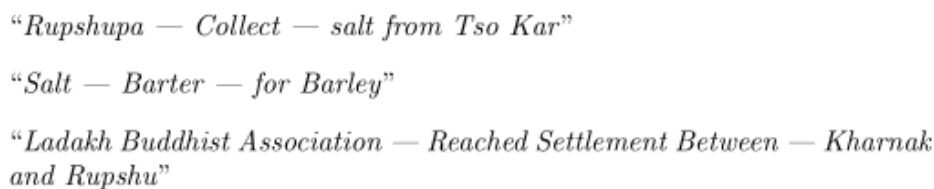
The allowed relationships are a carefully chosen set relevant to the research domain:

- `trades_with`, `exchanges_for`, `extracts_from`, `transports_via`
- `taxes`, `regulates`, `governs`, `controls`, `disputes`, `licenses`, `monopolizes`
- `supplies`, `depends_on`, `connects_to`, `migrates_through`
- `administers`, `negotiates_with`

The LLM’s output is checked — relations that don’t match the allowed list are discarded, and results are further filtered by confidence score. Identical triples found in multiple places in the text are **weighted** by frequency: a relationship mentioned ten times is considered much stronger evidence than one mentioned once.

The processing is also parallelized: multiple entity pairs are analyzed simultaneously to speed things up, since this stage can be slow when processing a long book.

Output: A `weighted_knowledge_graph.csv` file — a table where every row is a Subject–Relation–Object triple, along with how often it was found and a piece of evidence text from the original source.



“Rupshupa — Collect — salt from Tso Kar”

“Salt — Barter — for Barley”

“Ladakh Buddhist Association — Reached Settlement Between — Kharnak and Rupshu”

Figure 4: Examples of Extracted Relationship Triples

3.6 Stage 6: Keeping the Graph Clean — Entity Type Correction

What happens: After a new book is processed, its results are merged into the larger, accumulated knowledge graph. At this point, the same entity might appear across multiple books — “Tibet”, for example, might have been classified slightly differently by TextRazor in different texts. This stage reviews all the entities in the cleaned, aggregated graph and cross-references them against all the NER results collected so far to assign the most accurate and consistent **type** (Person, Location, Group, Commodity, Concept) to each.

It also generates a review spreadsheet (`entity_type_review.csv`) where the researcher can manually inspect entities whose type is uncertain — especially those that have never been manually confirmed. The researcher can flag corrections directly in this file.

If a researcher has already made manual corrections (stored in `config.py`), those corrections are preserved and not overwritten.

Output: An updated `cleaned_entities.json` file with corrected entity types, and an `entity_type_review.csv` file for human review.

3.7 Stage 7: Building and Visualizing the Graph — Regenerating the HTML

What happens: The final stage takes the fully cleaned, corrected, and weighted knowledge graph and turns it into an **interactive visual network** that can be opened in any web browser.

Each entity becomes a **node** in the graph. The type of entity determines its shape and color:

- Locations appear as one color and shape

- People as another
- Commodities as another
- Groups and Concepts each have their own visual style

The size of each node reflects how **central** it is to the network — entities that appear in many relationships are shown as larger nodes.

Each relationship between entities becomes an **edge** (a line connecting two nodes), labeled with the type of relationship. Heavier edges indicate relationships that were found more frequently in the texts.

The user can interact with the visualization: zoom in and out, click nodes to see details, filter by entity type or relationship type, and explore the network’s structure visually.

Output: A `network_visualization.html` file saved in the `outputs/` directory, which can be opened in any browser to explore the full knowledge graph.

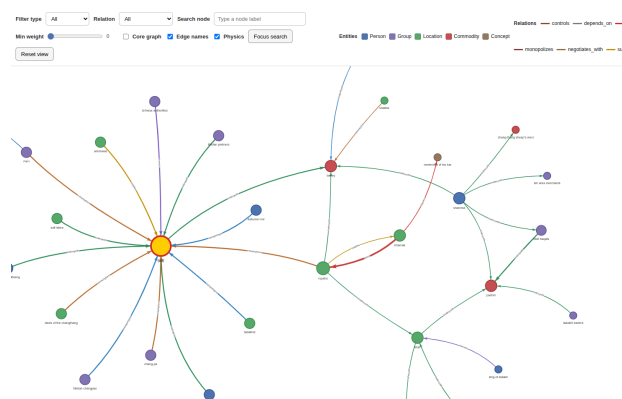


Figure 5: Macro View of Core Network Graph



Figure 6: Close-up View of the Segment Centralized Around Salt

4 Human-in-the-Loop Design

The pipeline is deliberately not fully automatic. Two critical checkpoints require human judgment:

Checkpoint 1 — Topic Selection: After topics are extracted, the researcher reviews them and selects only the relevant ones. This ensures the co-occurrence analysis is focused on historically meaningful entities and not distracted by incidental mentions.

Checkpoint 2 — Entity Type Review: After entity types are assigned by the automated system, a review spreadsheet is generated. The researcher can inspect uncertain cases and make corrections, which are then permanently stored in the configuration file and applied to future runs.

This hybrid approach — automation for scale, human judgment for quality — is intentional. Historical research requires interpretive expertise that no AI can fully replicate. The pipeline amplifies what a researcher can do, rather than trying to replace their judgment.

5 What the System Produces

At the end of processing any book, and after the graph is updated, the researcher has access to:

Output	Description
<code>ner_results.txt</code>	All named entities and topics found in the book, with scores
<code>entity_cooccurrences.txt</code>	Which entities appeared near each other, how often, and in what context
<code>weighted_knowledge_graph.csv</code>	All extracted relationships as structured Subject–Relation–Object triples
<code>cleaned_entities.json</code>	The full entity list with verified types, accumulated across all books
<code>cleaned_network.graphml</code>	The full graph in a standard format for further analysis
<code>network_visualization.html</code>	An interactive browser-based visual exploration of the knowledge graph
<code>entity_type_review.csv</code>	A spreadsheet of entities pending human type verification

6 Key Design Choices and Their Rationale

Why TextRazor? TextRazor provides entity linking — it doesn’t just find names, it identifies *which* person or place a name refers to, using a structured knowledge base. This is far more powerful than simple name detection and provides the Freebase category hierarchy used for entity typing.

Why a local LLM (Llama 3 / Ollama)? Research data — particularly unpublished historical sources — should not be sent to external commercial AI services unless necessary. Using a locally-hosted LLM keeps the research data private and avoids costs associated with large-volume API calls.

Why a constrained relation set? Allowing the LLM to invent arbitrary relation labels would produce an inconsistent, unanalyzable graph. By specifying a small, domain-appropriate set of allowed relations, every edge in the final graph is comparable and meaningful within the research framework.

Why co-occurrence before relation extraction? Running the full LLM analysis on every possible entity pair in a long book would be computationally prohibitive. The co-occurrence stage acts as a filter, pre-selecting only the pairs that actually appear near each other in the text and are therefore worth investigating for explicit relationships.

Why human curation of topics? Automated NER over historical texts will pick up many entities that are irrelevant to the research question. A historian’s judgment about which topics matter — which geographical regions, which commodities, which types of social actors — cannot be automated without significant loss of precision.

7 Limitations and Considerations

- **Text quality:** The pipeline works best with clearly digitized, text-layer PDFs. Scanned image PDFs would require OCR pre-processing, which can introduce errors.
- **LLM accuracy:** Llama 3 is powerful but not infallible. It can occasionally misidentify relationships, hallucinate connections, or miss nuance. The confidence threshold and evidence requirement mitigate this, but human review of key findings is always advisable.

- **TextRazor entity linking:** The Freebase knowledge base underlying TextRazor was frozen around 2016. Highly specialized historical terms, place names in regional transliterations, or obscure persons may not be recognized.
- **Co-occurrence window size:** The ± 50 word window is a reasonable heuristic but is not perfect. Very long sentences or paragraph structures could either include unrelated entities or exclude related ones.
- **Scale:** Processing a long book through all stages — particularly the LLM relation extraction — can take significant time. The parallel processing option helps, but large-scale corpora require patience.

8 Conclusion

This pipeline represents a practical and principled approach to **computational historical research**. It does not attempt to replace the historian — it amplifies what a historian can do. Tasks that would take years of manual reading and note-taking (identifying every mention of salt taxation in a corpus of fifty books, for example) can be completed in hours, leaving the researcher free to do the interpretive, analytical, and argumentative work that is the core of historical scholarship.

The knowledge graph that the system builds is not just a visualization — it is a **structured, queryable record** of historical relationships, grounded in primary sources, weighted by frequency of evidence, and systematically organized. It can be used to trace the centrality of specific commodities in a trade network, identify the key brokers and intermediaries in a frontier economy, or track how governance and taxation patterns changed across different regions and time periods.

The combination of cloud-based entity recognition (TextRazor), locally-hosted language understanding (Llama 3), sliding-window co-occurrence analysis, and human-in-the-loop curation makes this pipeline both powerful and responsible — leveraging the best of modern AI tooling while keeping the historian in control of the research process.